

CombiBERT: Choosing the Best Layers Combination in Sentence-BERT Model Variant to Reduce Losses in Knowledge Distillation

Willy Lau
Master of Computer Science
Binus Graduate Program
Jakarta, Indonesia
willy.lau@binus.ac.id

Sani Muhamad Isa
Master of Computer Science
Binus Graduate Program
Jakarta, Indonesia
sani.m.isa@binus.ac.id

Abstract—The Bidirectional Encoder Representations from Transformers (BERT) model has revolutionized natural language processing (NLP) tasks, achieving state-of-the-art results across various domains. However, its deep architecture, consisting of 12 to 24 layers, presents significant challenges for deployment in real-time applications and resource-constrained environments due to its high computational and memory requirements. This research introduces CombiBERT, an alternative layer reduction approach using brute force combinatorial to select the essential encoder layer to retain while minimizing losses in knowledge distillation on Sentence-BERT model variant by systematically evaluating all possible combinations of layers to identify the most efficient configurations. By leveraging this exhaustive method, the study aims to maintain semantic similarity performance while reducing computational complexity. Experiments were conducted on the 12-layer stsb-bert-base model using datasets such as All-NLI, Wikipedia, and STS Benchmark. The evaluation metrics included Mean Squared Error (MSE) and Spearman correlation of cosine similarity. The results identified optimal configurations, including a 6-layer model ([0,5,7,9,10,11]), which achieved the lowest MSE of 0.019820 and a Spearman correlation within 0.54% of the original model. Additionally, a 4-layer model ([0,4,10,11]) was identified, which balanced performance and efficiency, achieving an MSE of 0.040428 and a Spearman correlation within 1.12% of the teacher model. This study highlights the potential of systematic layer reduction in enhancing the efficiency of transformer models. The proposed method provides a practical pathway for deploying BERT in real-world applications, such as recommendation engines and semantic search tools, without significant loss in performance.

Keywords—Layer Reduction, Model Compression, Knowledge Distillation, BERT, Sentence-BERT

I. INTRODUCTION

In recent years, Bidirectional Encoder Representations from Transformers (BERT) has emerged as a key model in natural language processing (NLP), achieving state-of-the-art results in various tasks such as question answering, sentiment analysis, and machine translation [1], [2], [3], [4], [5]. BERT's architecture, with its multi-layer bidirectional Transformer encoder, allows for deep contextual understanding of text. However, the model's depth, typically 12 to 24 layers, leads to significant computational overhead, making it less feasible for deployment in real-time systems or on devices with limited resources. Reducing the number of layers without compromising performance is a critical challenge for many practical applications [6].

In this research, layer reduction method has been proposed to distill Sentence-BERT variant models by retaining only the most essential encoder layers. Similar approaches to compress BERT have been explored in various studies, demonstrating

the potential of layer reduction and knowledge distillation for efficiency without substantial performance loss [7], [8], [9], [10], [11], [12]. Inspired by the brute force method in password cracking [13], this paper introduces an alternative approach that uses a combinatorial brute force method to select the best n-layers to retain from the Sentence-BERT model variant. By systematically exploring all possible combinations of layer subsets, this approach ensures that the identified n-layer will have the minimum knowledge distillation losses.

This brute force method with combinatorial optimization is characterized by its exhaustive evaluation of all combinations of n-layers, which guarantees finding the best-performing subset. While computationally intensive, this strategy is crucial for identifying key layers that contribute disproportionately to the model's semantic understanding, thereby achieving optimal performance with reduced computational complexity. The methodology hypothesizes that certain layers in Sentence-BERT variant are more critical than others and selecting these layers strategically can lead to substantial efficiency gains without significant degradation in semantic similarity.

This paper will describe the theoretical foundation for this method, the experimental setup for evaluating layer selection, and the empirical results obtained through this approach. By employing this combination-based optimization strategy, this research highlights an alternative method to minimize losses in knowledge distillation.

II. RELATED WORKS

A. Transformers

Transformers are deep learning models that revolutionized NLP and machine learning, introduced in the research paper "Attention is All You Need" by [14]. Unlike RNNs or CNNs, transformers rely on self-attention mechanisms, enabling them to capture relationships between words regardless of their positions. This structure makes them highly effective for sequential tasks like translation, with an encoder-decoder architecture consisting of multiple self-attention layers and feed-forward networks [15], [16], [17].

B. BERT

BERT (Bidirectional Encoder Representations from Transformers) is a multi-layer bidirectional transformer encoder proposed by [1] that captures contextual information effectively for NLP tasks like sentiment analysis and question answering. It uses masked language modeling (MLM) during pre-training, where some words are masked and predicted based on their surrounding context. BERT's ability to generate rich contextual representations allows it to outperform previous models across many NLP benchmarks.

C. DistilBERT

DistilBERT, introduced by [18] is a smaller, faster version of BERT, designed to work in resource-constrained environments. It leverages knowledge distillation, where the smaller student model learns to mimic the behavior of the larger BERT model while retaining most of its language understanding capabilities [6]. Key modifications include:

- **Layer Reduction:** Reduces the number of layers to cut computational complexity.
- **Parameter Distillation:** Learns to match BERT's outputs on MLM and NSP tasks.
- **Efficient Training:** Uses specialized loss functions and regularization to maintain performance with fewer parameters.

DistilBERT provides a compact alternative to BERT with faster inference and lower memory usage, making it ideal for real-world applications.

III. PROPOSED METHOD

The hypothesis in this research was proposed during analyzing the first distillation experiment results with various *stsb-bert-base* layer combinations. In our findings, there are some combinations that produced the best results in Spearman Correlation between the distilled model similarity metrics scoring and STS Benchmark dataset label scoring always contains both layer 10 and 11. Which triggered us to assume, there may be best n-layer to retain.

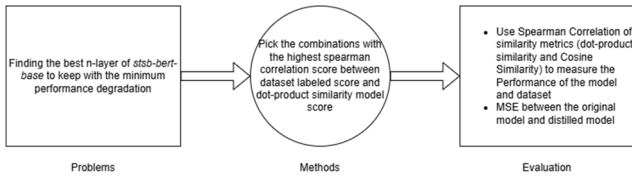


Fig. 1 Research Hypothesis

The proposed research methodology mainly focused on reducing the layers of the Sentence-BERT model variant by selecting the optimal n-Layer of *stsb-bert-base* in terms of computational efficiency while preserving semantic similarity performance. The model distillation methodology can be seen in fig. 2. Below is a step-by-step outline of the approach:

A. Dataset Preparation

Three datasets were used for training and evaluation:

- **All-NLI:** The All-NLI dataset is a combination of multiple natural language inference datasets, such as SNLI and MNLI, sourced from the Hugging Face Dataset Hub. This dataset was already pre-split into training and development sets, simplifying preprocessing tasks. The main preprocessing step involved removing duplicate sentences to ensure data quality and prevent redundancy in training. After preprocessing, the dataset was encoded into vector representations using the *stsb-bert-base* model from the sentence-transformers library. This embedding step ensured consistency and compatibility across datasets during training.
- **Wikipedia:** The Wikipedia dataset also sourced from the Hugging Face Dataset Hub. The data was used for unsupervised training and required more extensive

preprocessing. Unlike the All-NLI dataset, it was not pre-split into training and development sets. The dataset was manually split, with 5000 samples reserved for development and the rest used for training. After splitting, duplicate sentences were removed, and the dataset was encoded using the *stsb-bert-base* model.

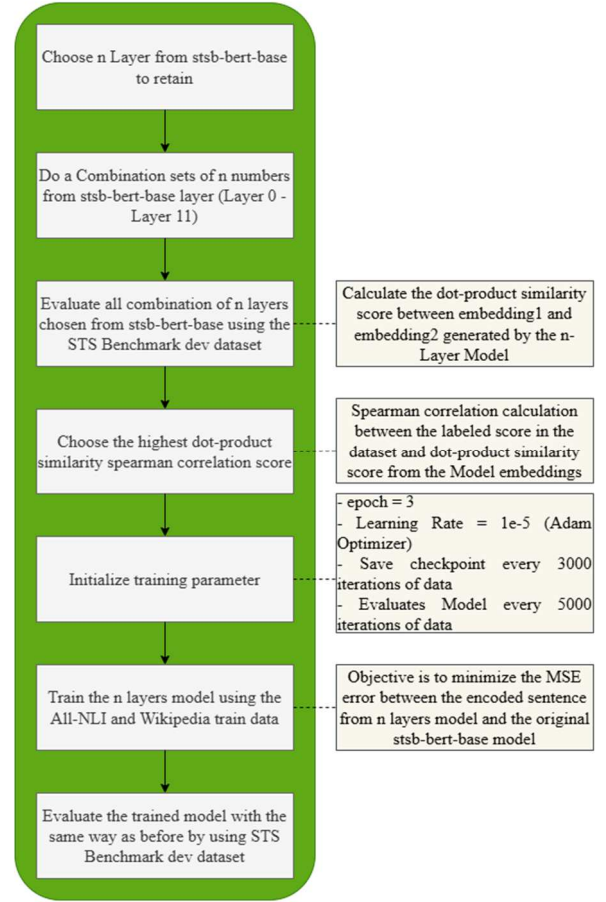


Fig. 2 Methodology of Distillation Procedure

- **STS Benchmark:** This dataset is reserved only for evaluating semantic similarity correlation and pre-split development data comprising 1500 samples was selected. As this dataset was smaller and already pre-split, preprocessing was minimal, focusing on filtering and embedding sentences using the *stsb-bert-base* model [19].

Train Data Construction

The All-NLI train subset and the Wikipedia train subset were concatenated to form a unified training dataset. This combined dataset provided a rich and diverse set of sentence pairs, capturing a wide range of linguistic structures and contexts. The final training corpus consisted of almost 9.1 million sentence pairs, with the majority sourced from the All-NLI dataset.

Evaluation Data Construction

For the Mean Squared Error (MSE) evaluation during training, the 5000 development samples from Wikipedia and random 5000 samples from the All-NLI development set were concatenated, creating a dataset of 10,000 samples. This combined dataset was used to evaluate the ability of the reduced models to approximate the outputs of the teacher model.

- **Quora Question Pair**
The Quora Question Pair dataset is a widely recognized

benchmark in natural language processing (NLP) research, specifically designed for tasks involving semantic similarity and duplicate detection. It was introduced by Quora to address the challenge of identifying semantically similar or duplicate questions within its repository. The ability to detect duplicate questions not only enhances the user experience by reducing redundancy but also improves search efficiency and content management on the platform. The dataset composition information is as follows:

- **404,290** question pairs in total.
- Each pair includes the following fields:
 - qid1**: Unique ID for the first question in the pair.
 - qid2**: Unique ID for the second question in the pair.
 - question1**: Text of the first question.
 - question2**: Text of the second question.
 - is_duplicate**: A binary label indicating whether the two questions are duplicates (**1 for duplicate, 0 for non-duplicate**).

Despite its utility, the dataset presents several challenges. Some question pairs are ambiguous, making it difficult even for humans to classify them confidently, introducing potential noise in the labels. As suggested by [20], there might be some labels that were labeled incorrect, suggesting subjectivity in the labeling. Furthermore, as the dataset is derived from Quora, models trained on it may show limited generalization when applied to other domains with different characteristics or question formats.

Due to the imbalanced data distribution, we must do an upsampling of the minority data in this research. This is achieved by resampling the minority class (**is_duplicate = 1**) to match the number of instances in the majority class (**is_duplicate = 0**). The `resample` function from the `sklearn.utils` module is used for this purpose.

The upsampled minority class is then combined with the majority class to form a new balanced dataset. After this step, the combined dataset is shuffled to ensure the model does not learn any patterns related to the order of data points. The shuffling process helps maintain the random distribution of classes during training, thus avoiding any unintentional bias.

After upsampling the minority class, the final dataset size increased to 510084 entries. To prepare the data for model evaluation, the dataset is divided into training and testing sets, with 20% of the data, amounting to 102017 entries, allocated for testing. The remaining 80% of the data, consisting of 408067 entries, is used for training the model. This split ensures a balanced evaluation, providing sufficient data to train the model while reserving an adequate portion for testing its generalization performance.

B. Layer Selection Method

The goal of this research is to identify a subset of n layers from the 12-layer *stsb-bert-base* (fig. 3) model that maximizes computational efficiency while retaining high semantic similarity performance. To achieve this, a brute force combinatorial approach is employed. This method ensures the identification of the optimal subset of layers, albeit at a higher computational cost due to its exhaustive nature. The process involved:

1. **Choosing n Layers**: n Layers are selected from the original 12 layers (Layer 0 to Layer 11) of the *stsb-bert-base* model.
2. **Combinatorial Layer Selection**: All possible combinations of n layers from the available layers are generated. These combinations were evaluated to determine which subset of layers preserves the model's semantic understanding.

Brute Force Method Explanation

The brute force method is a straightforward algorithmic approach where all possible solutions or configurations are enumerated and evaluated based on a specific criterion [13], [21]. In this context, all combinations of n layers from the 12-layer *stsb-bert-base* model are generated and assessed. This **guarantees that the optimal layer configuration can be found**, as no potential combination is overlooked. While computationally intensive, this method is particularly suited for tasks where exhaustive exploration is necessary to understand the contribution of each layer.

For the 12-layer *stsb-bert-base* model, the number of combinations for selecting n layers is given by the binomial coefficient:

$$\binom{12}{n} = \frac{12!}{n!(12-n)!} \quad (1)$$

For example:

- For $n = 4$, $\binom{12}{4} = 495$ Combinations
- For $n = 6$, $\binom{12}{6} = 924$ Combinations

Each of these untrained layer combinations were evaluated during inference using metrics such as the **Spearman correlation score** and **Mean Squared Error (MSE)** to determine its ability to preserve semantic similarity. This exhaustive search provides key insights into the relative importance of individual layers and their combinations in contributing to the model's performance which assesses the alignment between the outputs of the reduced model and the full *stsb-bert-base* model.

Trade-Offs and Justification

Although the brute force method is computationally expensive, it was crucial for this research because:

1. It guaranteed the identification of the optimal layer subset, eliminating the risk of missing better-performing configurations.
2. It offers a systematic way to analyze the contributions of individual layers to Sentence-BERT's overall performance.
3. The results from the brute force evaluation serve as a baseline for future studies exploring more computationally efficient layer selection methods.

C. Evaluation of Layer Combinations

For each combination of n layers, the model was evaluated using the **STS Benchmark** dev dataset:

- **Evaluation Metric**: The Spearman correlation score of similarity metrics (dot-product similarity and Cosine Similarity) were used to evaluate each combination of layers. This score measures the correlation between the **STS Benchmark dataset label score and the dot-product similarity score generated by model embedding**.
- The layer combination with the highest **Spearman correlation score of dot-product similarity** is selected

for further training since **dot-product similarity represent the unnormalized form of cosine similarity**. These evaluation methods are the same as in [22] except in this research we were using the Spearman Correlation. Spearman correlation was used because the STS-B dataset cannot be considered linear as the relationship between sentences can be complex, involving nuances such as context, word choice, and syntactic structures.

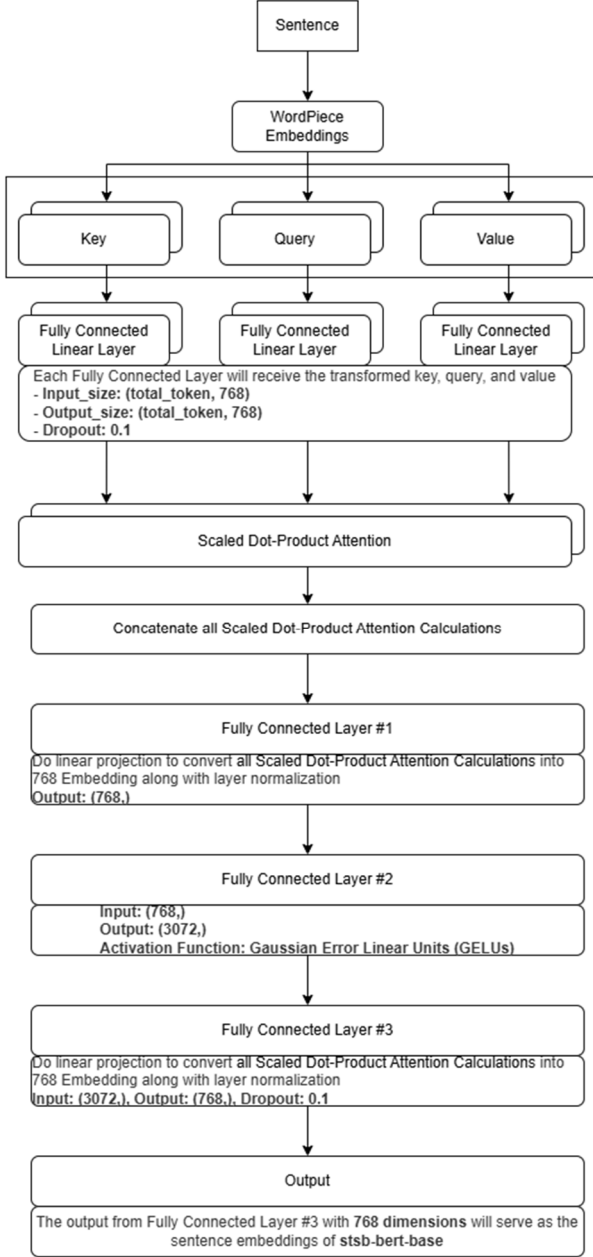


Fig. 3 Detailed Decomposition of one *stsb-bert-base* Layer

The evaluation of layer combinations was a critical step in determining the optimal subset of layers that preserve semantic similarity while reducing computational complexity. To facilitate the evaluation of a large number of combinations efficiently, the sentence-transformers Python library was employed. This library integrates the GPU acceleration in Pytorch which implements parallel processing, significantly speeding up the evaluation process. By leveraging the computational power of NVIDIA GPU, the library allowed for the concurrent encoding of sentences and

similarity evaluations, drastically reducing the time required for processing the datasets.

The parallel processing capability of the sentence-transformers library was particularly beneficial given the computational intensity of the brute force combinatorial approach, which involved generating and testing hundreds of layer combinations. For instance, the evaluation of 495 combinations for $n = 4$ layers and 924 combinations for $n = 6$ layers was completed efficiently, enabling comprehensive exploration within a feasible timeframe.

D. Model Training

Once the optimal n layers were selected, the following training steps were performed:

1. **Parameter Initialization:** Training parameters, such as learning rate, optimizer, and epochs, were initialized. Specifically:
 - Learning Rate = $1e-5$
 - Epoch = 3
 - Batch Size = 64
 - Adam optimizer was used for optimization.

The learning rate of $1e-5$ was selected based on prior studies that suggest this value is optimal for fine-tuning transformer models [23], [24]. Similarly, the batch size of 64 was chosen to balance memory efficiency and convergence speed, considering the GPU's available VRAM.

2. **Training:** The n layers selected in the previous step were trained using the **All-NLI** and **Wikipedia** training datasets. The training objective was to minimize the mean squared error (MSE) between the sentence embeddings generated by the reduced-layer model and those from the full *stsb-bert-base* model based on [23] approach on fig 3. The Mean Squared Error (MSE) loss was chosen for its ability to directly quantify the difference between sentence embeddings of the reduced model and the full *stsb-bert-base* model.

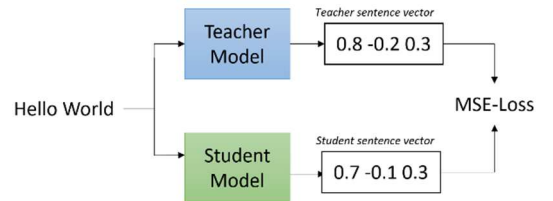


Fig. 4 Approach on Model Distillation [23]

- Checkpoints were saved every 3000 iterations to ensure that intermediate model states could be evaluated and preserved. This approach allowed for early identification of promising configurations and served as a safeguard against potential interruptions during training.
- The model was evaluated every 3000 iterations to monitor performance. During training, the model was evaluated every 3000 iterations using the **ALL-NLI** and **Wikipedia** development dataset to determine the MSE between the teacher model and student model. These evaluations provided real-time feedback on the model's ability to retain semantic similarity, guiding adjustments to the learning rate and batch size if necessary.

E. Final Model Evaluation

The final model, trained with n layers, was evaluated using **STS Benchmark** dev dataset to compute the Spearman correlation score and assess the semantic similarity performance. The method of evaluation was the same as in Evaluation of Layer Combinations section which implements the parallel processing. The results were analysed to compare the performance of the reduced-layer model against the full *stsb-bert-base* model.

F. SNN-CNN Hybrid Architecture for Binary Classification task on Quora Question Pair Dataset

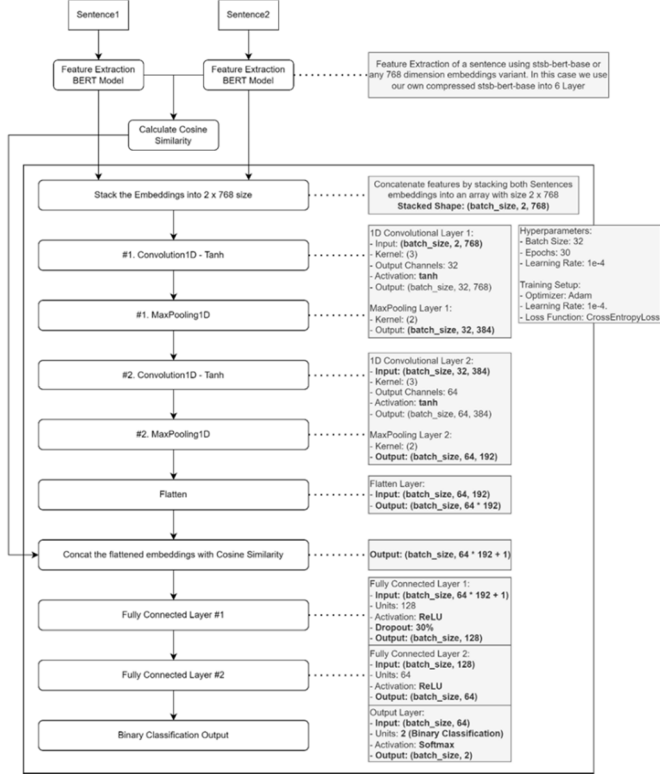


Figure 5 SNN-CNN Hybrid Neural Network Architecture

For further testing the distilled model capability in a text classification we decided to test it in a quora question pair text classification task. For this task we will use our own crafted SNN-CNN hybrid architecture. This Siamese Neural Network (SNN)-CNN hybrid model is designed for binary classification tasks, specifically to determine whether two input sentences are semantically similar. The architecture begins by extracting features from two input sentences, Sentence1 and Sentence2, using a pre-trained BERT-based Sentence Transformer. This feature extraction generates embedding for each sentence, with each embedding being a vector of size 768, which captures the semantic meaning of the sentence in a dense and compact form.

By applying convolutional operations, the model effectively expands the dimensionality and complexity of the stacked features to improve the model learning capability. The first convolutional layer applies to a 1D convolution with a kernel size of 3 and 32 output channels, followed by a max-pooling layer with a kernel size of 2, which reduces the sequence length by half. The second convolutional layer further processes the features using a similar configuration but with 64 output channels. Another max-pooling layer is applied, resulting in a final feature map of shape (**batch_size**,

64, **192**). These convolutional layers, combined with the **hyperbolic tangent (tanh)** activation function, allow the model to learn nuanced patterns in the data while expanding the representation for subsequent layers.

The choice of the **tanh** activation function is particularly important in this architecture because it outputs values in the range **(-1, 1)**, which aligns with the value range of cosine similarity. Additionally, tanh ensures minimal distortion or loss of information from the embeddings generated by BERT, as the original BERT embeddings also have values that lie within the range **(-1, 1)**. In contrast, using an activation function like **ReLU**, which clips negative values to zero, would result in a significant loss of information contained in the embeddings, particularly for features that are negatively correlated.

The output of the convolutional layers is flattened into a single vector of size **64 × 192**, which represents the learned features of both sentences. This flattened vector is then concatenated with the cosine similarity score, ensuring that both local feature patterns and a global similarity measure are included in the classification process. The combined vector is passed through a series of fully connected layers. The first layer reduces the feature size to **128** units with a **ReLU** activation function and applies dropout with a **30%** rate to prevent overfitting. The second fully connected layer further reduces the feature size to **64** units, also using **ReLU** activation.

Finally, the output layer performs binary classification, mapping the 64-dimensional features to 2 output units, corresponding to the two classes (e.g., duplicate or not duplicate). A **softmax** function is applied to produce a probability distribution over the two classes, enabling the model to output confidence scores for each class. The model is trained using the Adam optimizer with a learning rate of **1e-4**, and the **CrossEntropyLoss** is used as the loss function to handle the binary classification task effectively. CrossEntropyLoss is well-suited for binary classification tasks when the output layer consists of two nodes representing the two classes. This loss function calculates the difference between the predicted probability distribution and the true class labels by penalizing incorrect predictions more heavily when the model is highly confident in its wrong predictions. By doing so, CrossEntropyLoss encourages the model to produce probabilities that are as close as possible to the actual class labels, ensuring better performance in distinguishing between duplicate and non-duplicate sentence pairs.

IV. EXPERIMENT RESULTS AND ANALYSIS

A. Experiment Setup

The experimental environment used in this research was PC-side operating system Ubuntu from WSL2 Windows, CPU AMD Ryzen 5 3600, GPU NVIDIA GeForce RTX 3060 12GB, 32GB of RAM, programming language Python 3.12.4. The process of distillation in this experiment used the Sentence-Transformers 3.1 Python library with prerequisite requirements of PyTorch 2.4.1 installed with CUDA 12.4 and Transformers 4.42.2 library. Sentence-Transformers specialize in sentence embedding within BERT models. According to [24], sentence embeddings generated by this framework outperform other models and achieve state-of-the-art performance.

This experiment employed a layer reduction technique to distill the *stsb-bert-base* model, which consists of 12 layers with an embedding size of 768. The goal was to produce two distilled models by selecting specific layers to retain, while keeping the embedding size unchanged. The experiment will output two types of distilled models:

- 4-layer combination model
- 6-layer combination model

These distilled models aim to balance performance and efficiency, maintaining meaningful representations with fewer layers. By applying the proposed method at Model Layer Selection section, this study identified combinations such as [0, 5, 7, 9, 10, 11] for 6-layer model and [0, 4, 10, 11] for 4-layer model. These findings demonstrated the potential of systematic layer reduction in enhancing the efficiency of transformer encoder models without significant performance degradation.

B. Results Analysis

The training process of the reduced models was conducted on the *stsb-bert-base* model using datasets such as All-NLI and Wikipedia. Both the 4-layer and 6-layer configurations were trained with the objective of minimizing the Mean Squared Error (MSE) between their embedding output and those of the full *stsb-bert-base* model. Training parameters included a learning rate of $1e-5$, a batch size of 64, and the use of the Adam optimizer to ensure stable convergence.

During training, the MSE dropped drastically when iterating through the first ~ 2 million training samples, demonstrating the models' rapid adaptation to the task. Unfortunately, the specific intermediate values from this early stage were not included in the research report but highlighted significant improvements in semantic similarity retention during the initial training phase. This observation aligns with the model's ability to quickly approximate key semantic relationships before plateauing in later iterations.

TABLE I MSE BETWEEN EMBEDDINGS GENERATED BY STSB-BERT-BASE AND DISTILLED MODELS

Layer Combination	MSE ($\times 100$)	Trainable Parameters
Teacher (stsb-bert-base)	-	109,482,240
[1, 4, 7, 10]	4.5102	52,779,264
[1, 5, 9, 11]	3.2838	52,779,264
[2, 4, 8, 10]	4.5223	52,779,264
[0, 4, 10, 11] (Combination Method)	3.0631	52,779,264
Odd Layers (1, 3, 5, ...)	3.7573	66,955,008
Even Layers (0, 2, 4, ...)	2.3752	66,955,008
[0, 5, 7, 9, 10, 11] (Combination Method)	1.9820	66,955,008

The evaluation of different layer combinations of the *stsb-bert-base* (layer 0 – layer 11) model highlighted the

performance trade-offs in reducing layers to improve computational efficiency. Cosine similarity, dot-product similarity, and Spearman correlation serve as key metrics, showing how well the distilled models align with the original model's outputs. Mean Squared Error (MSE) quantifies the average squared difference between the embeddings generated by the student and teacher models, with lower MSE indicating better semantic preservation. The 6-layer model exhibited a slower overall convergence rate but achieved superior final performance metrics, with the lowest MSE recorded at 0.019820. On the other hand, the 4-layer model converged faster, thanks to its reduced complexity, but exhibited slightly higher MSE, indicating a trade-off between efficiency and accuracy. **MSE values were scaled by 100 for readability**, for more detailed MSE results can be referred to table 1 while the detailed results of Spearman Correlation can be seen at table 2.

In this overall experiment, the best 4-layer was [0, 4, 10, 11] and the best 6-layer was [0, 5, 7, 9, 10, 11], while other combinations in the experiment results were randomly chosen. Based on the correlation results, the layers combination method proposed in this research shows that the models perform better than the randomly chosen layers model. The best 4-layers model was [0, 4, 10, 11] with **1.13229%** cosine similarity difference from the base model. For the best 6-layers obtained from the combination method was [0, 5, 7, 9, 10, 11] with **0.541443%** percentage difference compared to the base model.

TABLE II SIMILARITY CORRELATION BETWEEN MODELS AND DATASET (BETTER IF RESULTS CLOSER TO 1)

Layers / Spearman	Cosine-Similarity	Dot-Product-Similarity
Teacher (stsb-bert-base)	0.8704	0.8461
[1, 4, 7, 10]	0.8590	0.8242
[1, 5, 9, 11]	0.8606	0.8326
[2, 4, 8, 10]	0.8581	0.8223
[0, 4, 10, 11] (Combination Method)	0.8606	0.8327
Odd Layers (0, 2, 4, ...)	0.8653	0.8309
Even Layers (1, 3, 5, ...)	0.8648	0.8382
[0, 5, 7, 9, 10, 11] (Combination Method)	0.8657	0.8393

The results show that not all layers in *stsb-bert-base* contribute equally to the final output. By strategically selecting certain layers (such as [0, 5, 7, 9, 10, 11]), the student model can perform closely to the teacher model with fewer layers, leading to a lighter model with minimal performance degradation. This finding is valuable for applications requiring more efficient models without sacrificing much in accuracy. Furthermore, the even-layer combination (MSE(x100): 2.3752) significantly outperforms the odd-layer combination (MSE(x100): 3.7573), suggesting that certain key representations lie in the even layers of the *stsb-bert-base* architecture and layer selection that contains layer 10 and layer 11 shows better results than other layers that do not contain those layers.

TABLE III MSE BETWEEN EMBEDDINGS GENERATED BY STSB-ROBERTA-BASE-V2 AND DISTILLED MODELS

Layer Combination	MSE ($\times 100$)	Trainable Parameters
Teacher (stsb-roberta-base-v2)	-	124,645,632
[0, 7, 10, 11] (Combination Method)	4.2804	67,942,656
[0, 1, 7, 9, 10, 11] (Combination Method)	2.6779	82,118,400

For further testing, we also applied our distillation method in *stsb-roberta-base-v2* that is another Sentence-BERT model variant based on RoBERTa to see how the results look like. Due to the limited time, we only managed to train two best 4-layer and 6-layer model. For the 6-layer model we decided to train it until $\sim 4.5M$ of data because training beyond that the MSE keeps increasing. Based on table III, we can see that the MSE between the embeddings from the combination method models and the base model shows very little difference. The distilled 4-layer model shows a higher MSE than the 6-layer due to less layer configuration.

Using the combination method on *stsb-roberta-base-v2* produced a very small gap at spearman correlation of cosine-similarity between the combination method produced models and the teacher model. Based on the results in table IV, the difference in spearman correlation of cosine-similarity 4-layer distilled model was **0.91887%** from the base model while 6-layer distilled model was **0.111607%** percentage difference compared to the teacher model. In the spearman correlation of dot-product similarity, the 6-layer model produced a better results than the teacher model with 0.8553 compared to 0.8544. That result doesn't mean that the distilled model outperforms the teacher model since the dot-product similarity is an unnormalized form of Cosine Similarity.

TABLE IV SIMILARITY CORRELATION BETWEEN MODELS AND DATASET (BETTER IF RESULTS CLOSER TO 1)

Layers / Spearman	Cosine-Similarity	Dot-Product-Similarity
Teacher (stsb-roberta-base-v2)	0.8965	0.8544
[0, 7, 10, 11] (Combination Method)	0.8883	0.8469
[0, 1, 7, 9, 10, 11] (Combination Method)	0.8955	0.8553

By using our proposed method, we successfully produced knowledge distillation results within our expectations. To test whether the model can run text classification task properly we will use the distilled models as feature extraction in the text classification task. The results of text classification on quora question pair dataset can be seen on table V and table VI.

From Table VI, we observe that the **base model (stsb-RoBERTa-base-v2)** achieved slightly higher accuracy (**91.15%**) compared to the **distilled model (90.79%)** in table V. The distilled model showed **higher precision for the Not Duplicate class (0.9347 against 0.9321)** but at the cost of a small drop in recall (**0.8773 against 0.8876**). For the Duplicate class, the teacher achieved better precision (**0.8927 against 0.8843**), whereas the distilled model achieved

stronger recall (**0.9386 against 0.9353**). Both models had **comparable F1-scores**, showing that the distilled model retained most of the predictive performance of its teacher.

TABLE V QUORA QUESTION PAIR DATASET CLASSIFICATION RESULTS USING DISTILLED 6L stsb-RoBERTa-base-v2

30 Epoch(s)	Precision	Recall	F1-Score
0 (Not Duplicate)	0.9347	0.8773	0.9051
1 (Duplicate)	0.8843	0.9386	0.9107
Accuracy			0.9079
Confusion Matrix			
	0 (Not Duplicate)	1 (Duplicate)	
0 (Not Duplicate)	44767	6261	
1 (Duplicate)	3130	47859	

The results also shows that the teacher model correctly classified **45,295 non-duplicate pairs** and **47,691 duplicate pairs**. However, it misclassified **5,733 non-duplicates** as duplicates and **3,298 duplicates** as non-duplicates. This aligns with its higher precision for non-duplicates, but slightly weaker recall compared to the distilled model while the distilled model correctly classified **44,767 non-duplicates** and **47,859 duplicates**. Compared to the teacher, it misclassified more non-duplicates (**6,261 against 5,733**) but fewer duplicates (**3,130 against 3,298**). This confirms the earlier observation that the distilled model is **more sensitive to detecting duplicates (higher recall)**, though it introduces slightly more false positives when classifying non-duplicates.

TABLE VI QUORA QUESTION PAIR DATASET CLASSIFICATION RESULTS USING stsb-RoBERTa-base-v2 as Feature Extraction

30 Epoch(s)	Precision	Recall	F1-Score
0 (Not Duplicate)	0.9321	0.8876	0.9093
1 (Duplicate)	0.8927	0.9353	0.9135
Accuracy			0.9115
Confusion Matrix			
	0 (Not Duplicate)	1 (Duplicate)	
0 (Not Duplicate)	45295	5733	
1 (Duplicate)	3298	47691	

The results overall demonstrate that the **distilled model effectively preserved the performance of the teacher model** on the Quora Question Pairs dataset, with only a marginal decrease in accuracy (0.36%). Interestingly, the distilled model even outperformed the teacher in recall for duplicates and precision for non-duplicates. This highlights the effectiveness of knowledge distillation in producing a lighter model while maintaining competitive performance in semantic similarity classification tasks.

V. CONCLUSION AND FUTURE RESEARCH

This research demonstrated the effectiveness of a combinatorial layer reduction method for distilling Sentence-BERT variant models. The proposed approach, which identified key subsets of layers, ensured a balance between model efficiency and semantic performance. Results indicated that not all layers contributed equally to *stsb-bert-base*'s output, and optimal combinations, such as [0, 5, 7, 9, 10, 11], offer substantial reductions in computational cost with minimal degradation in performance. The 6-layer model using this combination achieved the lowest Mean Squared Error (MSE) and the closest semantic similarity performance to the original *stsb-bert-base* model. Similarly, the 4-layer combination [0, 4, 10, 11] provided the best 4-layer model combinations than the rest. In terms of cosine similarity

divergence between distilled and base model, the distilled 6L stsb-RoBERTa-base-v2 gives out a very outstanding results with only 0.111607% percentage difference compared to the teacher model. We assume this can happen because the teacher model already came with a very good knowledge in semantic similarity. This layer selection strategy was crucial for applications that require lightweight, real-time models without significant loss in accuracy. However, it was essential to recognize that while the STS Benchmark provided a controlled environment for evaluating semantic similarity, real-life applications may yield different results due to varying data characteristics and operational constraints. Furthermore, the combination method used in this research was not recommended for implementation in large transformer encoder models with more than 12 layers, as the method would be quite time-consuming due to the complexity of the layers.

For future research, the proposed approach can be further evaluated and fine-tuned in real-life scenarios such as recommendation systems, or sentiment analysis tools for social media platforms. Another potential application is in search engines that rank and retrieve information based on semantic similarity, enabling more accurate and efficient search results. Implementing the reduced BERT models in these environments will help assess the practical gains in latency, resource consumption, and user experience [25]. Future work could also explore adaptive layer selection, where layers are dynamically activated based on task complexity or device limitations.

REFERENCES

- [1] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," *arXiv preprint arXiv:1810.04805*, 2018.
- [2] K. Clark, "What Does Bert Look At? An Analysis of Bert's Attention," *arXiv preprint arXiv:1906.04341*, 2019.
- [3] C. Sun, X. Qiu, Y. Xu, and X. Huang, "How to fine-tune bert for text classification?," in *Chinese computational linguistics: 18th China national conference, CCL 2019, Kunming, China, October 18–20, 2019, proceedings 18*, Springer, 2019, pp. 194–206.
- [4] Y. Liu *et al.*, "Roberta: A robustly optimized bert pretraining approach," *arXiv preprint arXiv:1907.11692*, 2019.
- [5] A. Wang, A. Singh, J. Michael, F. Hill, O. Levy, and S. R. Bowman, "GLUE: A multi-task benchmark and analysis platform for natural language understanding," *arXiv preprint arXiv:1804.07461*, 2018.
- [6] J. Gou, B. Yu, S. J. Maybank, and D. Tao, "Knowledge distillation: A survey," *Int J Comput Vis*, vol. 129, no. 6, pp. 1789–1819, 2021.
- [7] S. Sun, Y. Cheng, Z. Gan, and J. Liu, "Patient knowledge distillation for bert model compression," *arXiv preprint arXiv:1908.09355*, 2019.
- [8] Z. Lan, "Albert: A lite bert for self-supervised learning of language representations," *arXiv preprint arXiv:1909.11942*, 2019.
- [9] N. Houshy *et al.*, "Parameter-efficient transfer learning for NLP," in *International conference on machine learning*, PMLR, 2019, pp. 2790–2799.
- [10] Z. Sun, H. Yu, X. Song, R. Liu, Y. Yang, and D. Zhou, "Mobilebert: a compact task-agnostic bert for resource-limited devices," *arXiv preprint arXiv:2004.02984*, 2020.
- [11] X. Jiao *et al.*, "Tinybert: Distilling bert for natural language understanding," *arXiv preprint arXiv:1909.10351*, 2019.
- [12] G. Hinton, "Distilling the Knowledge in a Neural Network," *arXiv preprint arXiv:1503.02531*, 2015.
- [13] I. Alkhawaja *et al.*, "Password cracking with brute force algorithm and dictionary attack using parallel programming," *Applied Sciences*, vol. 13, no. 10, p. 5979, 2023.
- [14] A. Vaswani *et al.*, "Attention is all you need. arXiv 2017," *arXiv preprint arXiv:1706.03762*, vol. 3762, 2023.
- [15] P. Michel, O. Levy, and G. Neubig, "Are sixteen heads really better than one?," *Adv Neural Inf Process Syst*, vol. 32, 2019.
- [16] T. Lin, Y. Wang, X. Liu, and X. Qiu, "A survey of transformers," *AI open*, vol. 3, pp. 111–132, 2022.
- [17] T. B. Brown, "Language models are few-shot learners," *arXiv preprint arXiv:2005.14165*, 2020.
- [18] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter," *arXiv preprint arXiv:1910.01108*, 2019.
- [19] D. Cer, M. Diab, E. Agirre, I. Lopez-Gazpio, and L. Specia, "Semeval-2017 task 1: Semantic textual similarity-multilingual and cross-lingual focused evaluation," *arXiv preprint arXiv:1708.00055*, 2017.
- [20] L. Sharma, L. Graesser, N. Nangia, and U. Evci, "Natural language understanding with the quora question pairs dataset," *arXiv preprint arXiv:1907.01041*, 2019.
- [21] M. J. H. Heule and O. Kullmann, "The science of brute force," *Commun ACM*, vol. 60, no. 8, pp. 70–79, 2017.
- [22] D. Yang, D. Zhu, H. Gai, and F. Wan, "Semantic Similarity Calculating based on BERT," *Journal of Electrical Systems*, vol. 20, no. 2, pp. 73–79, 2024.
- [23] N. Reimers and I. Gurevych, "Making monolingual sentence embeddings multilingual using knowledge distillation," *arXiv preprint arXiv:2004.09813*, 2020.
- [24] N. Reimers and I. Gurevych, "Sentence-bert: Sentence embeddings using siamese bert-networks," *arXiv preprint arXiv:1908.10084*, 2019.
- [25] N. Kitaev, L. Kaiser, and A. Levskaya, "Reformer: The efficient transformer," *arXiv preprint arXiv:2001.04451*, 2020.